

天津大学

《前端开发技术实践》实验报告

第四组



学 院 智能与计算学部
专 业 软件工程
年 级 2023级
姓 名 戴茗静 曾意 贾思韵 高灿
学 号 3023244157 3023244158
 3023244168 3023208045

2025 年 6 月 10 日

一、项目需求分析

基于外卖服务的快速发展，本项目用于实现外卖订餐系统的核心闭环流程，重点满足消费者与商家的基础交互需求。用户端完成从注册登录、商家浏览、商品选购、订单支付到历史订单查看的完整链路；商家端实现信息注册、资质提交、商品上架及店铺信息维护能力。系统设计聚焦流程通畅性及数据准确性，实现了外卖的点餐送餐功能。

1.1 用户端功能需求分析

用户账户体系需保障安全性，通过手机号注册时强制密码复杂度校验（大小写字母混合且 ≥ 8 位），此设计直接服务于系统安全性需求。登录后，“我的”页面展示个人基础信息（昵称、手机号、性别）及头像（支持用户上传或默认显示），满足用户身份识别需求；退出登录功能则确保账户可控性。

首页作为流量入口，需高效引导用户决策。分类导航（美食/地方小吃/米粉面馆/品质套餐等）的设计旨在结构化呈现商家类型，可降低用户搜索成本；推荐商家列表（名称、评分、月销量、配送信息）需通过数据排序策略辅助用户选择，隐含平台运营需求。

商家详情页需同时承载信息传递与交易转化功能。应用需商家基础信息（图片、简介、起送费、配送费）建立用户信任，商品列表（名称、简介、价格、图片）需直观呈现选择项。购物车管理采用加减号交互实现商品数量调整，实时计算总价并与起送费比对，此机制直接服务于商家制定的交易规则——未达起送金额时禁用结算按钮，从流程上保障商家利益。

结算流程需兼顾效率与准确性。地址管理模块（增删改查+选择）满足用户多场景配送需求，其字段设计（详细地址、联系人、电话）直接关联配送履约可行性。订单明细展示（商品价格、配送费、总额）提供交易透明度，为支付决策提供依据。支付环节采用微信/支付宝双通道模拟，结果页展示支付时间并引导返回首页，形成操作闭环。“订单”页按支付状态分类（未支付/已支付），满足用户对交易记录的追溯需求。

1.2 商家端功能需求分析

商家注册需建立严格准入机制。手机号唯一性校验（一账号对应一商家）及前端实时反馈，旨在防止账号冲突并提升注册效率；信息完善页面（名称、地址、简介、费用、类型、图片）的字段设计，直接服务于平台对商家资质的标准化管理需求。

商家后台需提供持续运营能力。信息展示页面（图片、起送费、配送费、简介）帮助商家确认对外形象；信息修改功能支持动态调整经营策略（如费用变更）；商品上架流程（名称、简介、价格、图片）的设计需平衡操作效率与信息完整性——文本字段传递商品特征，图片提升吸引力，价格字段关联平台计价系统。商品提交后即时展示的要求，反映商家对经营灵活性的核心需求。

1.3 系统非功能性需求分析

前端采用Vue3框架需解决版本升级带来的兼容性挑战（如v-if/v-for渲染顺序调整），此选择服务于系统长期维护需求。密码加密存储是账户安全的基础保障，符合数据隐私规范。错误处理机制（如购物车访问undefined属性时用v-if拦截）直接影响用户体验稳定性，需通过防御性编码实现。核心业务流程（购物车计算、地址管理、支付跳转）的数据一致性，需依赖前后端状态同步策略，避免因数据延迟导致逻辑冲突（如起送费校验失效）。

二、项目设计

2.1 前端设计

2.2.1 登录

操作按钮：

个人/商家登录：选择以个人/商家身份登录

去注册：引导新用户先去注册

输入框：

输入用户名和密码

2.2.2 底部导航栏

首页：返回主界面

我的：包含用户的个人信息以及退出登录功能

订单：查看用户的订单

2.2.3 首页

顶部导航栏：

登录和注册按钮，让用户进行账户管理。

分类标签：

美食：包含各类食品选项。

地方小吃：提供地方特色小吃。

米粉面馆：提供米粉、面条、包子、粥、炸鸡、炸串等食品。

品质套餐：提供搭配齐全的套餐选项。

超级会员：提供会员服务，每月享受超值权益。

商家推荐：推荐商家列表，显示商家名称、配送信息等。

商家详情：显示商家的详细信息，如商家提供的各类商品的图片、价格、起送价、配送费等，可进行选购商品功能。

2.2.4 购物车信息

显示已加入购物车物品的名称、数量与价格。

2.2.5 订单信息

可查看未支付订单和已支付订单的明细。

确认订单：标明这是用户需要确认的订单信息。

配送信息：订单配送至：提示用户订单将被配送到的地址，具体的配送地址，收货人的联系电话。

商家信息：订单来自的商家名称和分店信息。

订单明细：用户所点的食品名称和数量，食品的价格，订单的配送费用。

支付按钮：用户点击此按钮进行支付。

订单总额：包括食品价格和配送费用的订单总金额。

2.2.6 支付信息

在线支付：标明这是用户进行在线支付的页面。

订单信息：提示用户这是他们即将支付的订单详情，包含商家及订单金额，该金额包括了食品价格和配送费用。

支付方式：可以选择微信支付或者支付宝支付。

支付按钮：用户点击此按钮以确认支付订单。

2.2.7 商家信息

商户地址：填写商家所在地。

商家简介：描述商家的定位。

商家图片：显示的商家图片。

配送费用：另需配送费。

起送金额：标明用户下单的最低金额要求。

店铺类型：选择商家在首页显示时的分类。

价格信息：食物的价格。

2.2 后端设计

2.2.1 接口设计

后端为前端提供数据支持，使用springboot框架，根据功能需求编写接口，以供前端调用。

- 商家

1. BusinessController / saveBusiness

参数：businessName , password

返回值：int (商家编号businessId)

功能：商家注册并返回代表商家的唯一编号作为账号

2. BusinessController / getBusinessByIdByPass

参数：businessId , password

返回值：int (影响的行数)

功能：商家登录

3. BusinessController / updateBusiness

参数：businessId , businessAddress ,

businessExplain , businessImg, starPrice , deliveryPrice ,
orderTypeId

返回值：int (影响的行数)

功能：更改（完善）商家的信息

4. FoodController / addFood

参数：businessId , foodName , foodExplain ,
foodImg , foodPrice

返回值：int (食品的标号foodId)

功能：商家可以上架自己的商品

5. BusinessController/listBusinessByOrderId

参数: orderId

返回值: business数组

功能: 根据点餐分类编号查询所属商家信息

6. BusinessController/getBusinessById

参数: businessId

返回值: business对象

功能: 根据商家编号查询商家信息

● 食品信息

1. FoodController/listFoodByBusinessId

参数: businessId

返回值: food数组

功能: 根据商家编号查询所属食品信息

● 购物车

1 CartController/listCart

参数: userId、businessId (可选)

返回值: cart数组 (多对一: 所属商家信息、所属食品信息)

功能: 根据用户编号查询此用户所有购物车信息; 根据用户编号和商家编号, 查询此用户购物车中某个商家的所有购物车信息

2 CartController/saveCart

参数: userId、businessId、foodId

返回值: int (影响的行数)

功能: 向购物车表中添加一条记录

3 CartController/updateCart

参数: userId、businessId、foodId、quantity 返回值: int (影响的行数)

功能: 根据用户编号、商家编号、食品编号更新数量

4 CartController/removeCart

参数: userId、businessId、foodId (可选) 返回值: int (影响的行数)

功能: 根据用户编号、商家编号、食品编号删除购物车表中的一条食品记录
根据用户编号、商家编号删除购物车表中的多条记录

● 送货地址

1 DeliveryAddressController/listDeliveryAddressByUserId

参数: userId

返回值: deliveryAddress数组

功能: 根据用户编号查询所属送货地址

2 DeliveryAddressController/getDeliveryAddressById

参数: daId

返回值: deliveryAddress对象

功能: 根据送货地址编号查询送货地址

3 DeliveryAddressController/saveDeliveryAddress

参数: contactName、contactSex、contactTel、address、userId

返回值: int (影响的行数)

功能: 向送货地址表中添加一条记录

4 DeliveryAddressController/updateDeliveryAddress

参数: daId、contactName、contactSex、contactTel、address、userId

返回值: int (影响的行数)

功能: 根据送货地址编号更新送货地址信息

5 DeliveryAddressController/removeDeliveryAddress

参数: daId

返回值: int (影响的行数)

功能: 根据送货地址编号删除一条记录

● 订单

1 OrdersController/createOrders

参数: userId、businessId、daId、orderTotal

返回值: int (订单编号)

功能: 根据用户编号、商家编号、订单总金额、送货地址编号向订单表中添加一条记录, 并获取自动生成的订单编号, 然后根据用户编号、商家编号从购物车表中查询所有数据, 批量添加到订单明细表中, 然后根据用户编号、商家编号删除购物车表中的数据

2 OrdersController/getOrdersById

参数: orderId

返回值: orders对象 (包括多对一: 商家信息; 一对多: 订单明细信息)

功能: 根据订单编号查询订单信息, 包括所属商家信息, 和此订单的所有订单明细信息

3 OrdersController/listOrdersByUserId

参数: userId

返回值: orders数组 (包括多对一: 商家信息; 一对多: 订单明细信息)

功能: 根据用户编号查询此用户的所有订单信息

4. OrdersController / payOk (新增)

参数: orderId

返回值: int (影响的行数)

功能: 改变订单属性, 将未支付0转成已支付1

● 用户

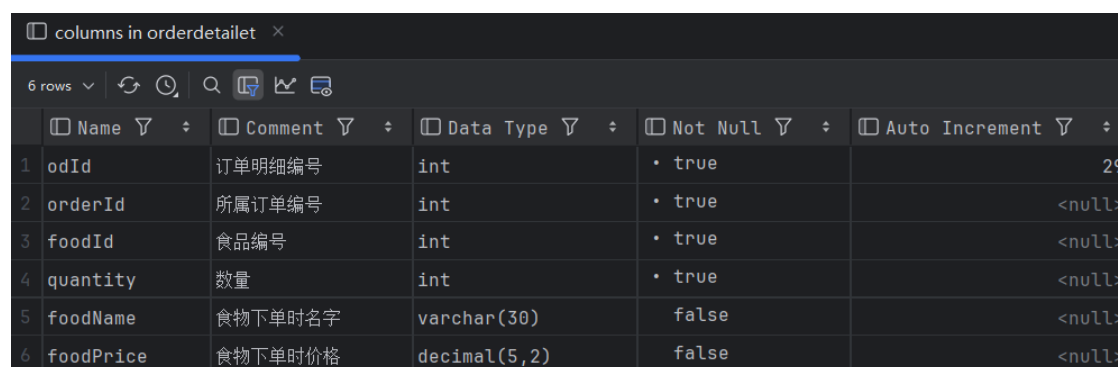
1 UserController/getUserByIdByPass
参数: userId、password
返回值: user对象
功能: 根据用户编号与密码查询用户信息

2 UserController/getUserById
参数: userId
返回值: int (返回行数)
功能: 根据用户编号查询用户表返回的行数

2.2.2 数据库改进

原有数据库表orderdetailt中的字段不够描述一个订单的详细信息, 对历史订单的商品列表中的食物名称、价格可能独属下单时的值, 这些值可能不同于当前表food中的值, 因为商家可以调整食物信息。

- 在表orderdetailt新增foodName、foodPrice字段
- 在表order中新增deliveryPrice字段



	Name	Comment	Data Type	Not Null	Auto Increment
1	odId	订单明细编号	int	• true	29
2	orderId	所属订单编号	int	• true	<null>
3	foodId	食品编号	int	• true	<null>
4	quantity	数量	int	• true	<null>
5	foodName	食物下单时名字	varchar(30)	false	<null>
6	foodPrice	食物下单时价格	decimal(5,2)	false	<null>

三、项目实现

3.1 Bug修复及功能完善

1. 解决商品信息和价格的修改不应影响历史订单的问题

订单展示页面每个订单的商品价格是通过动态实时调用数据库数据显示的, 造成修改商品价格或名称后历史订单的价格变成修改后的价格, 这显然不符合逻辑。

通过修改数据库和后端相应接口, 解决了该问题。

已支付订单信息：

万家饺子（软件园E18店） ▾	¥23
纯肉鲜肉（水饺） x2	¥20.00
配送费	¥3
小锅饭豆腐馆（全运店） ▾	¥26.5
蛋黄焗豆花 x3	¥24.00
配送费	¥2.5
万家饺子（软件园E18店） ▾	¥43
纯肉鲜肉（水饺） x2	¥40.00
配送费	¥3

2. 修复总价浮点数精度 bug

用字符串表示小数，然后通过字符串相加的算法，得到最终结果。

万家饺子（软件园E18店） ▾	¥18.33
纯肉鲜肉（水饺） x5	¥0.05
玉米鲜肉（水饺） x1	¥15.28
配送费	¥3

3. 解决购物车不能保存的问题

源项目框架中，当某一用户在商家中点餐将商品加入购物车但还未结算时，若返回、退出重进或在别的设备登陆时购物车不会同步信息。这是由于原本前端的实现中，加入购物车的操作并未立即调用后端接口更新数据库，退出重进时上一次的购物车数据丢失，没有被保存。

因此为了能保存未结算的购物车信息，在向购物车添加或移除商品时直接调用接口将更新同步到数据库，则下次前端再次访问购物车就能通过后端从数据库中获取上次购物车信息。

3.2 实现前端vue2到vue3的升级

在前端从vue2升级到vue3的过程中，我们首先引入了Composition API，它代替了原本的Option API，让代码逻辑更加灵活。比如在BusinessInfo组件里，原本分散在data、methods里的状态和函数现在能按照功能组织在一起。

其次，vue3把响应式系统从defineProterty换成了Proxy，这对购物车数量同步这类功能更加友好，比如Cart组件里直接操作数组或对象时不再需要Vue.set。

路由部分要适配新的useRouter和useRoute hooks，像订单跳转逻辑从this.\$router.push改成了更简洁的router.push。另外，全局属性挂载方式变了，之前挂在Vue.prototype上的axios和工具函数现在换成了app.config.globalProperties。同时，vue3的v-model也支持多个绑定，这在编辑地址表单时会更加方便。

最后，vue3的生命钩子名称变了，created换成了setup，但核心业务流程比如下单、支付这些基本不用变。

3.3 商家功能部分

3.3.1 实现商家账号注册，登录

天津大学北洋园校区

登录 注册

点击注册按钮，出现商家注册和个人注册两种注册方式，点击商家注册，跳转到商户注册页面，审核后进行登录：

← 请选择注册方式

商家注册

个人注册

首页 订单 我的

← 商户注册

手机号码: 15022639321

密码: *****

确认密码: *****

去审核

← 商家登陆

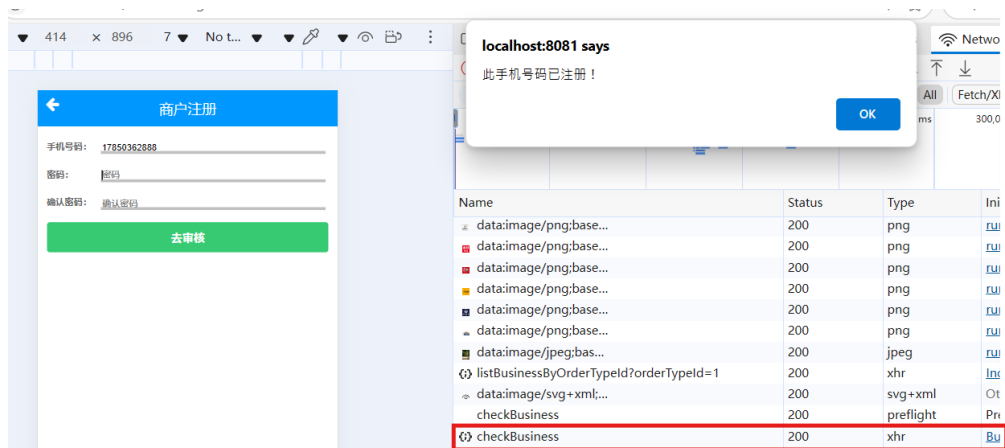
手机号码: 15022639321

密码: *****

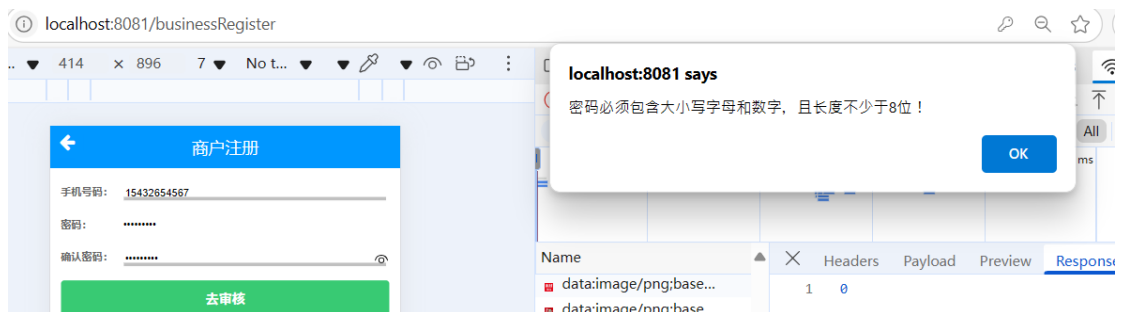
登陆

去注册

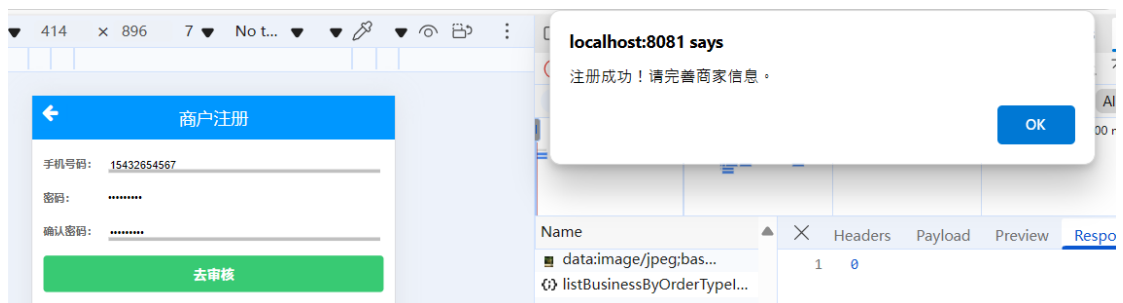
注册时需要输入手机号，限定一个手机号只能注册一个商家，当输入手机号时前端会调用后端检查手机号是否已被注册的接口，若已注册则有弹窗提示：



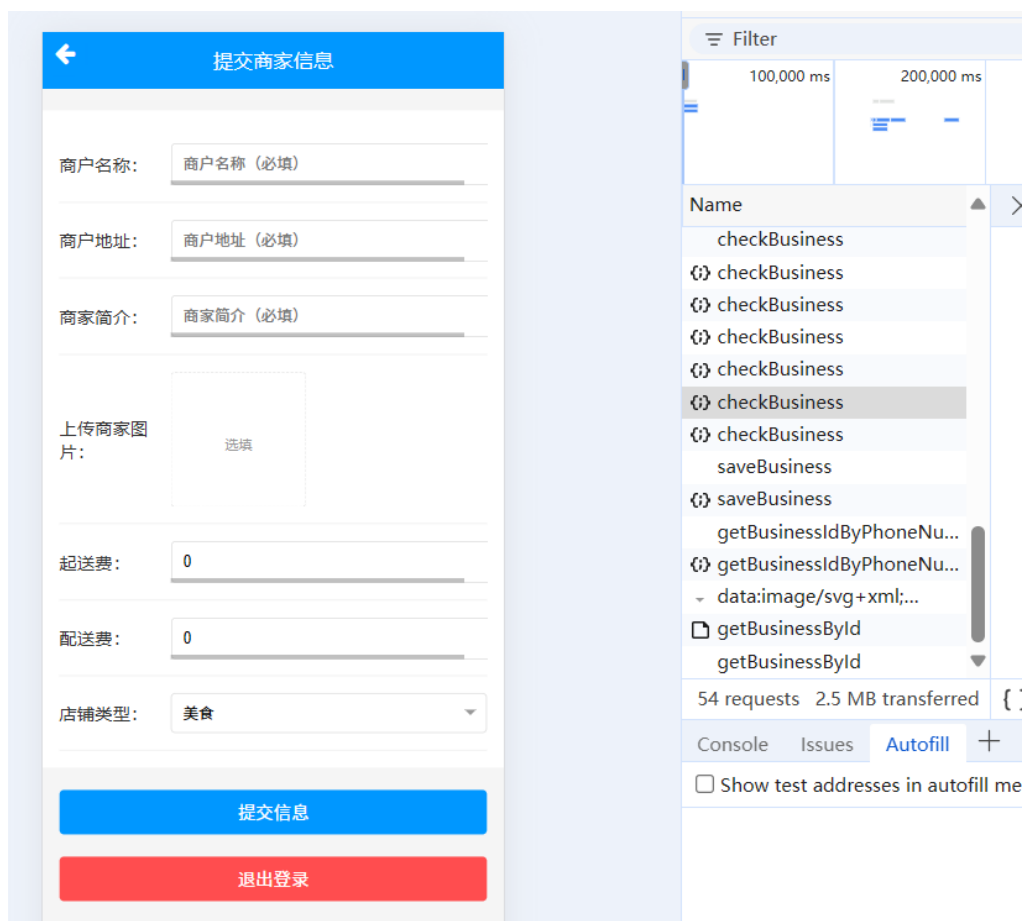
前端还会检测设置的密码是否安全性足够，要求密码必须包含大小写字母、长度不少于8位：



注册成功后，会跳转到要求商家完善信息的页面



跳转后：



3.3.2 商家信息与功能页面

1.展示商家信息

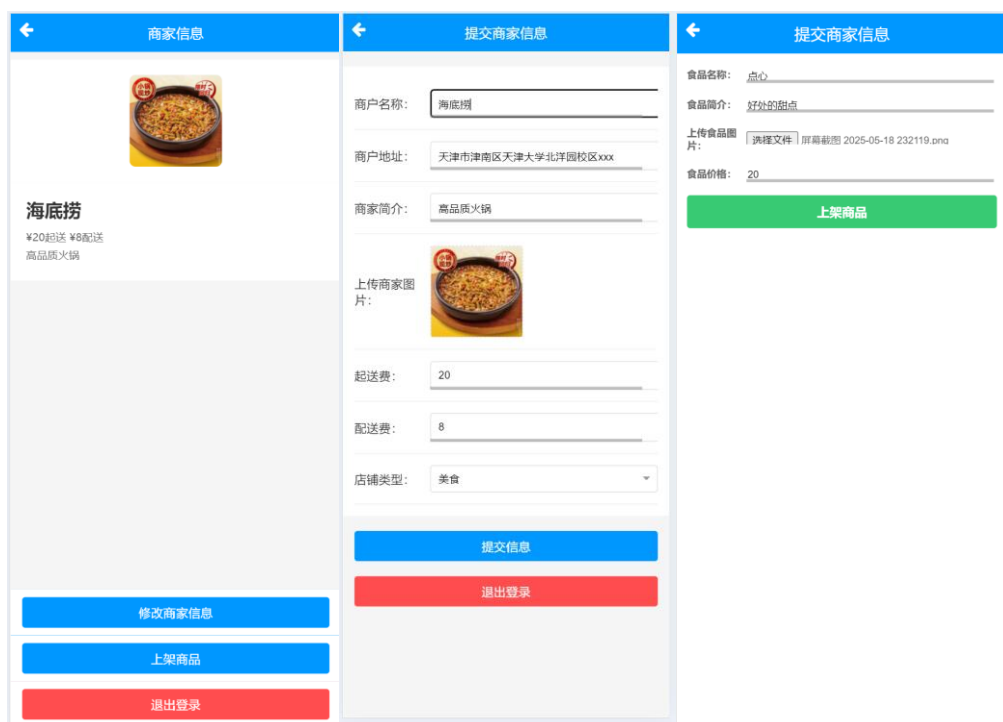
上方展示已提交的商家信息（商家图片、起送费、配送费、商家简介），下面展示“修改商家信息”按钮、“上架商品”按钮。

2.修改商家信息

点击“修改商家信息”按钮跳转到修改商家信息页面，商家可修改商户名称、商户地址、商家简介、起送费、配送费、店铺类型以及上传图片。

3.上架商品

点击“上架商品”按钮跳转到上架具体商品的页面，商家可以填写新商品的食品名称、食品简介、食品价格以及上传食物图片。

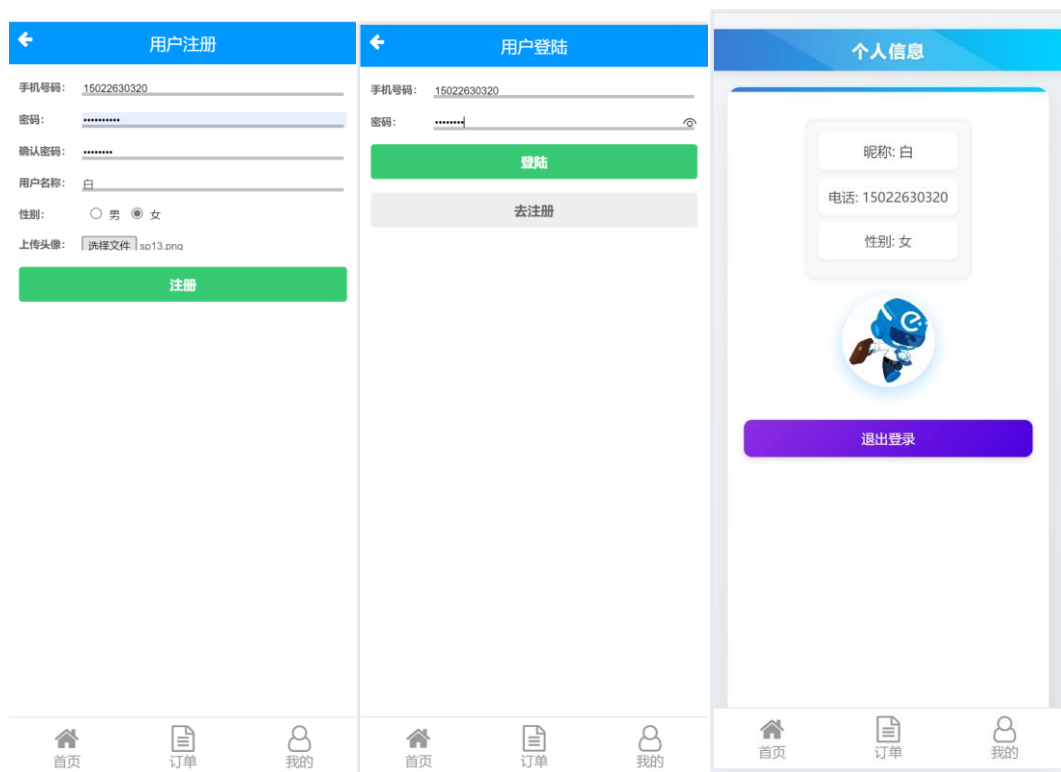


商家信息修改以及商家商品成功后会跳转回商家信息页面，即可看到更新过的商家信息和新上架的商品。

3.4 用户功能部分

3.4.1 实现“我的”页面

用户可以注册与登录，实现的功能与信息校验、密码设置规则同商家注册与登录。



“我的”页面：在“我的”页面中展示个人昵称、电话、性别，展示默认头像。

3.4.2 用户首页展示

首页展示商家分类、广告等信息。首页下方可滑动商家列表，默认展示美食类型商家。



可以选择特定类别的商家，点击相应图标会跳转至只展示该类别商家的页面并在其中下单等操作。

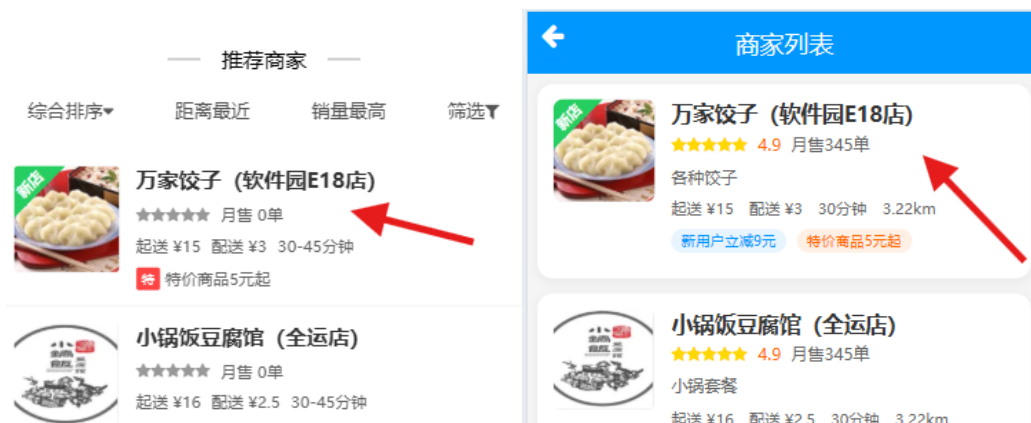


点击选择类别后：

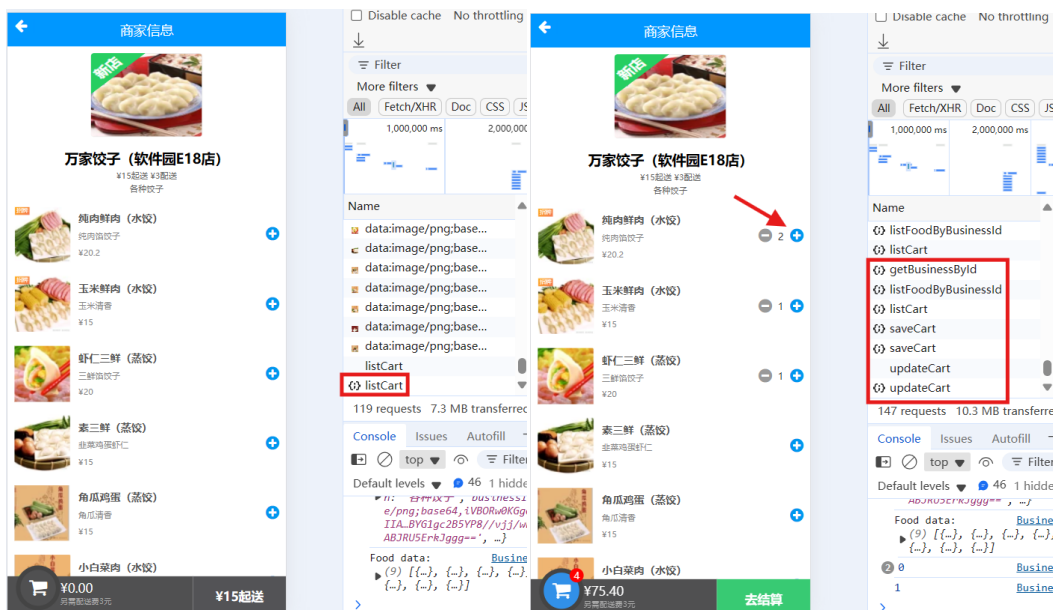


3.4.3 点餐界面

在首页或者选择类别后点击商家展示信息都可以进入该商家商品展示页面。



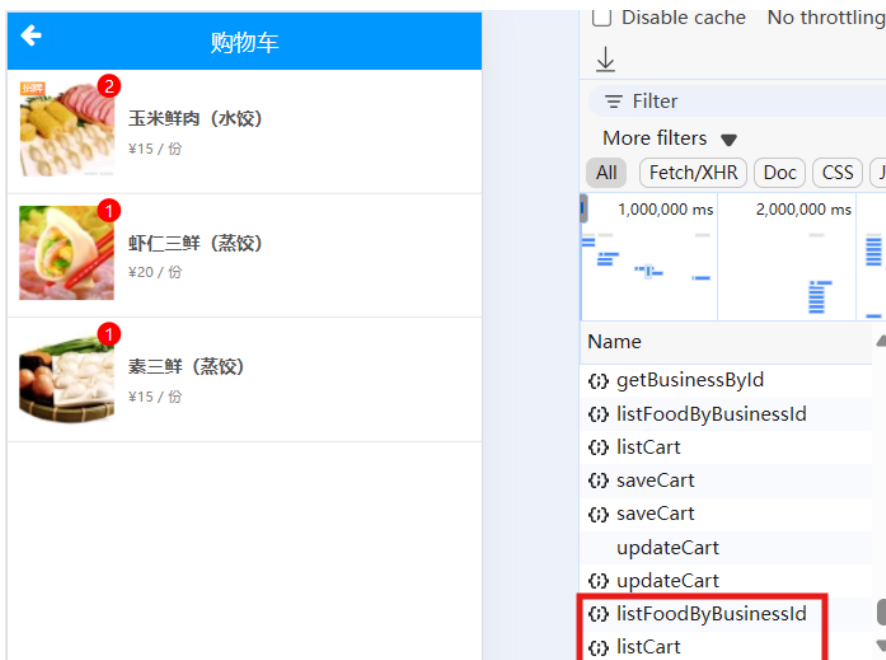
点击后展示商品列表，页面中展示该商家所有已上架商品的信息以及该商家配送费和起送费。该界面用于点餐，点击商品右侧加减号可以对购物车将指定数量该商品添加/移除。



可点击购物车图标查看已点商品

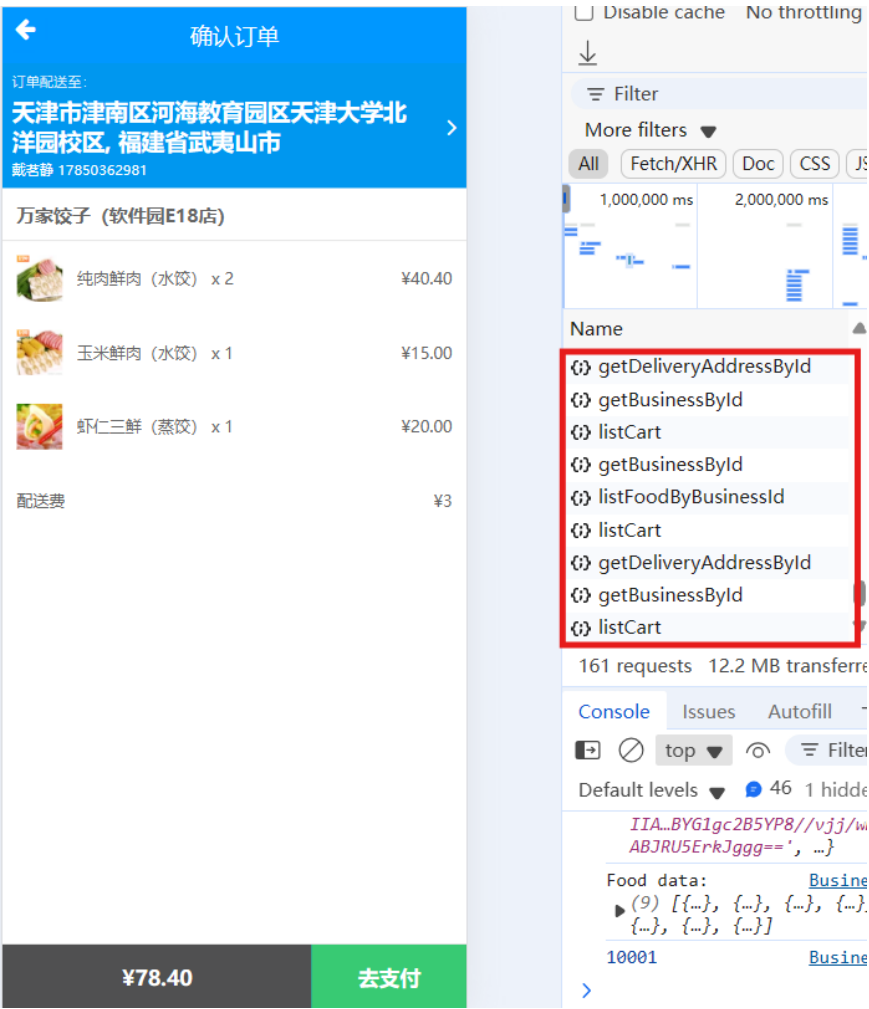


点击后跳转至购物车页面。该部分前端会调用后端查询商家配送信息的接口，将当前所选商品总价与之比较。选好商品后，点击“去结算”，可跳转至结算页面进行结算操作。

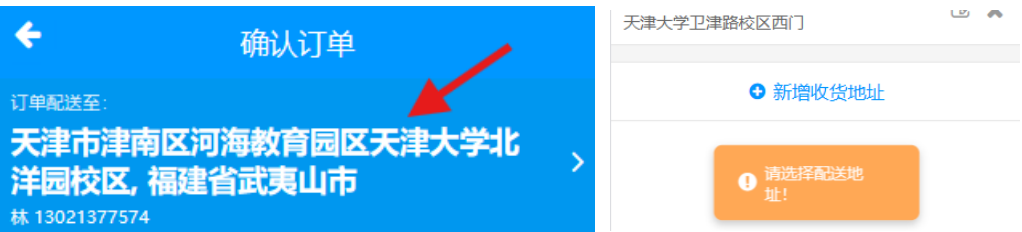


3.4.4 结算页面

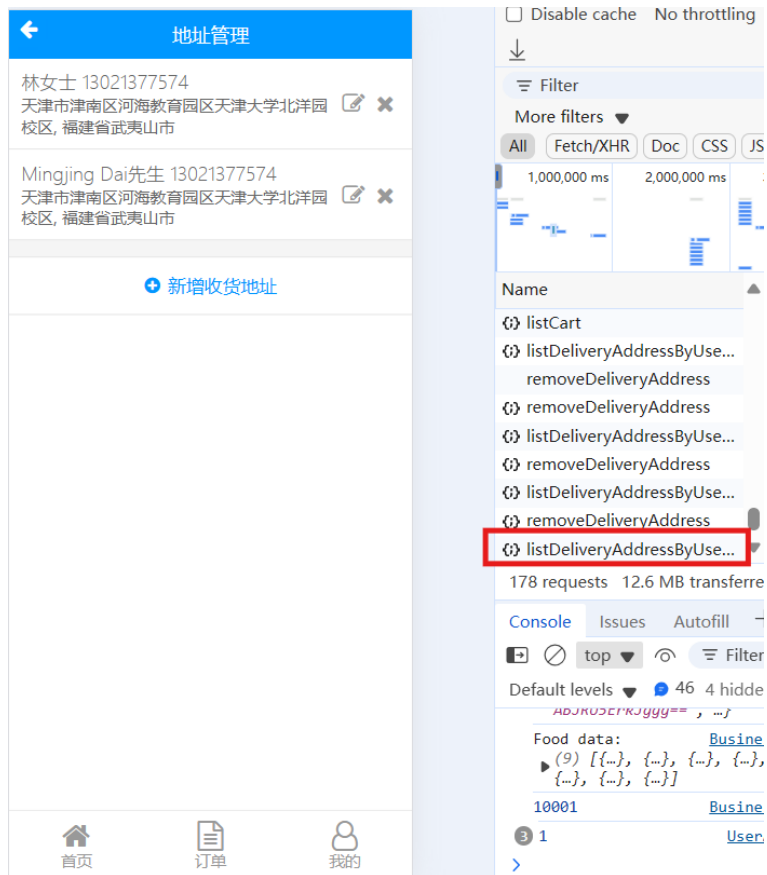
选择并展示配送信息，包括地址、下单人姓名、电话，对应下单的商家和费用明细。



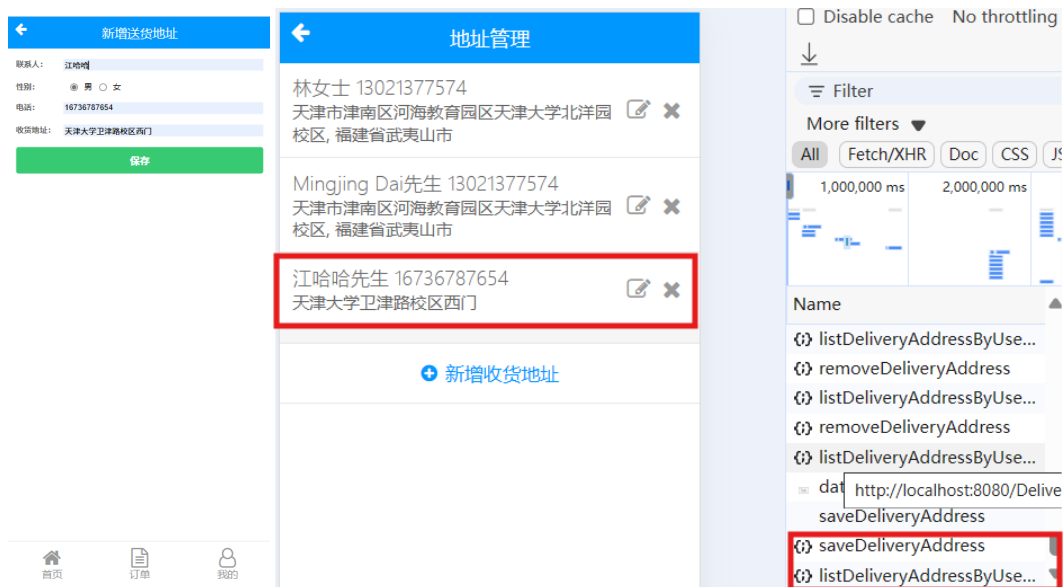
1. 可选地址



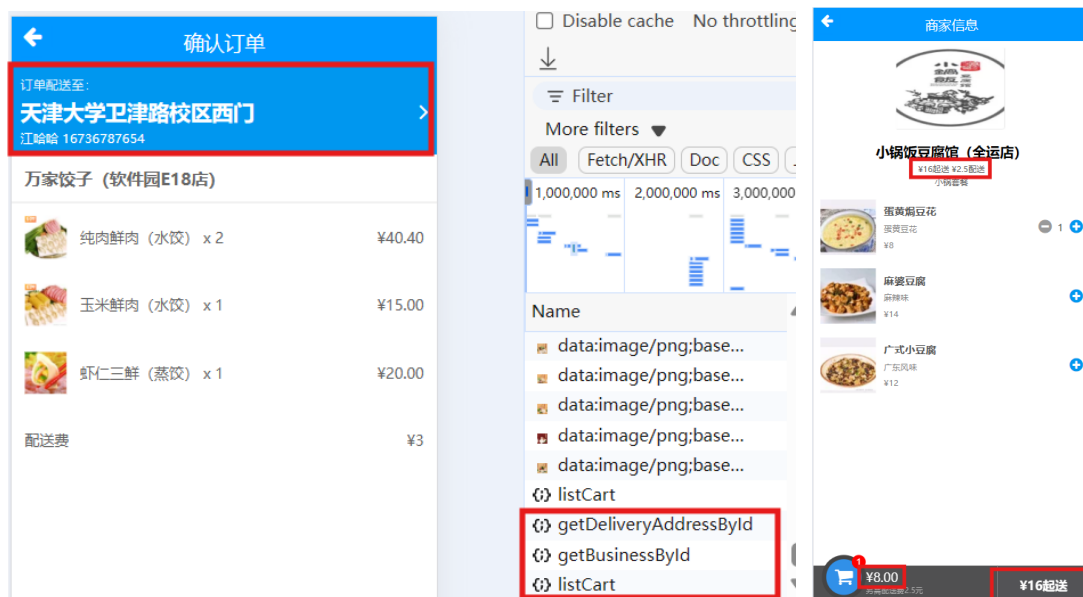
点击地址展示部分跳转至地址选择界面，该界面可删除/新增/修改/选择配送地址。



点击“新增收货地址”，输入地址信息保存后相应地址及时展示：



点击想要的地址，跳转回结算页面，配送地址会变成所选择的那项。



2. 将所有下单的商品结算，并附加配送费

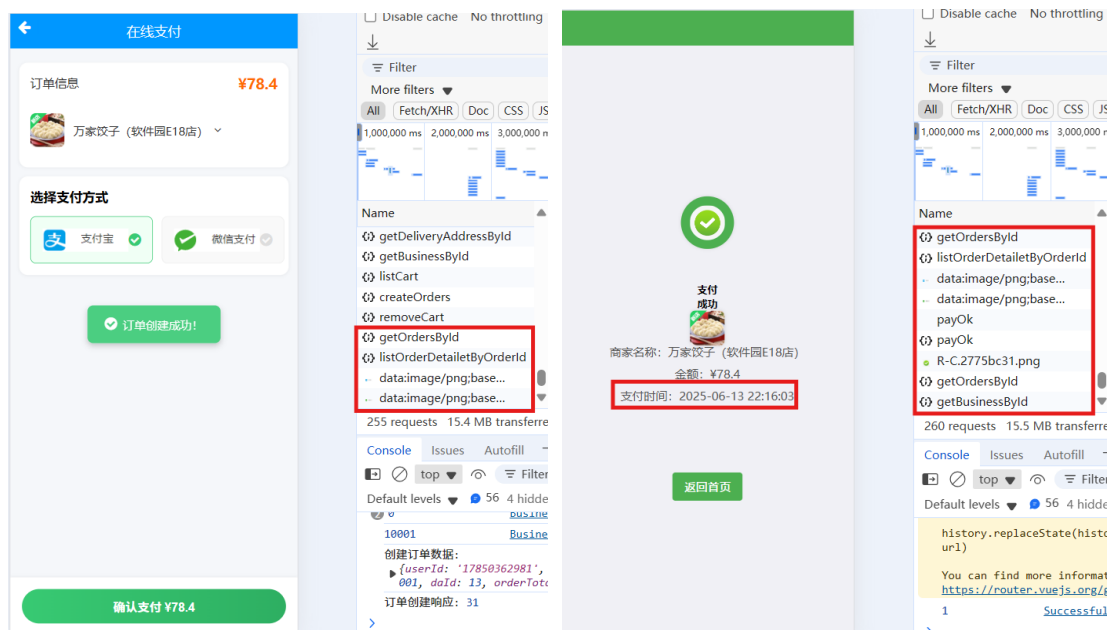


3. 商家设置起送费

当所选商品总价未达到起送费，左下角不显示去支付而是起送提醒。点击“去支付”，跳转至支付界面进行支付

3.4.5 支付界面

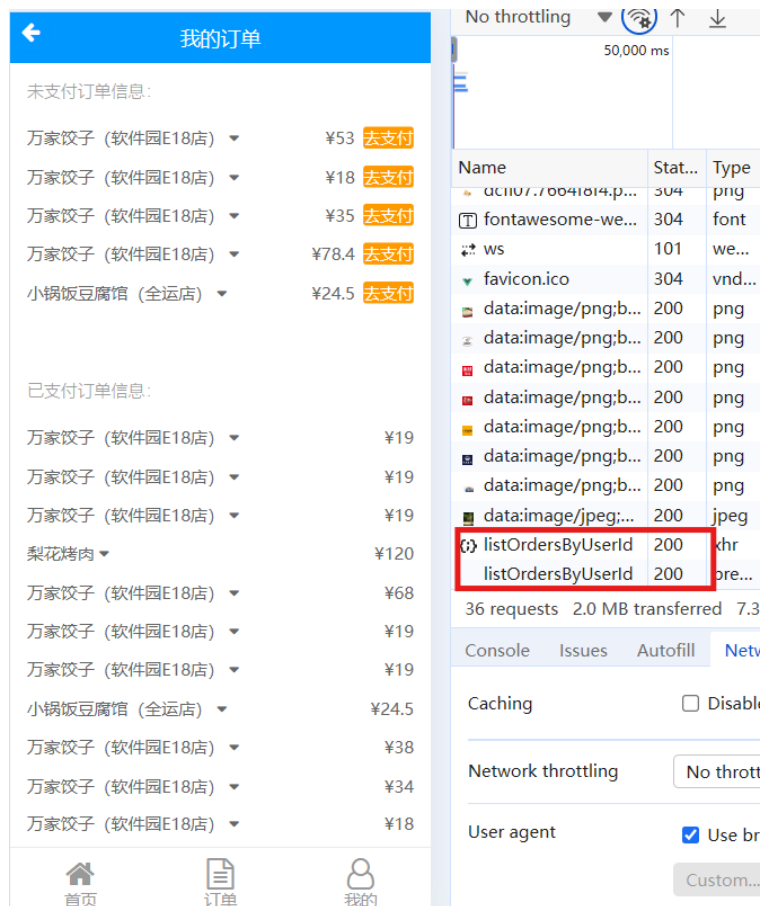
若未发生错误且数据库插入成功，则会短暂显示“订单创建成功”提示。页面显示需支付总价，并可选支付方式。



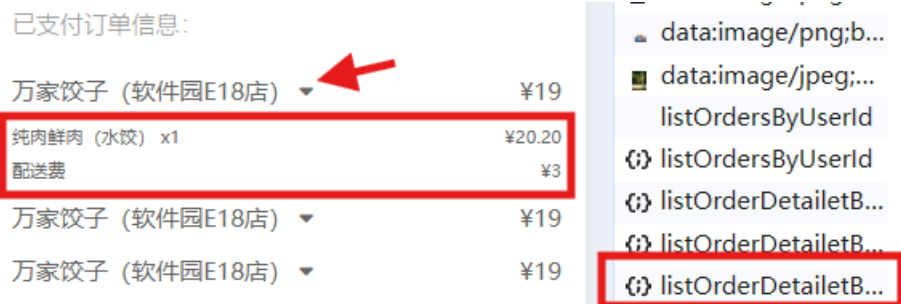
点击“确认支付后”，将跳转至支付结果页面，展示有支付完成时间，完成后可“返回首页”。

3.4.6 订单页面

点击页面下方菜单栏的“订单”，可以查看历史订单。展示不同状态的订单，对于未支付订单展示在页面上方，点击可以进入支付页面。



点击每条订单的下拉标识可以查看该订单的详细信息,包括该订单包含的商品信息。



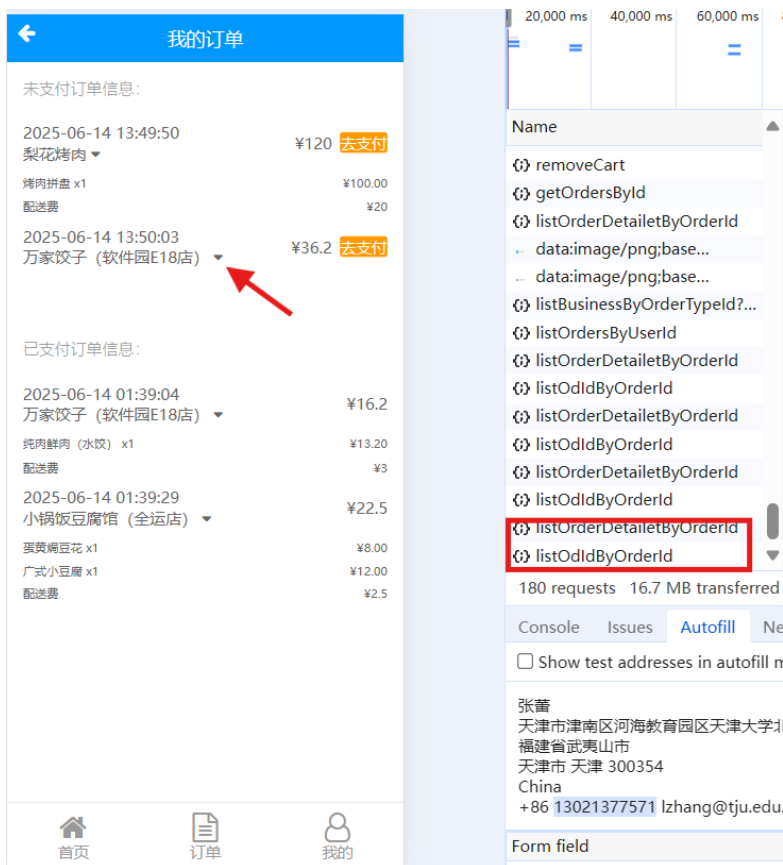
此处我们改进了数据库和后端的设计,对于原有问题:一个商家改变了某个商品的信息如价格后,用户中已完成的订单也会相应改变,不符合逻辑,情况如下:

已支付订单信息:	已支付订单信息:
万家饺子 (软件园E18店) ▼ ¥19	万家饺子 (软件园E18店) ▼ ¥19
纯肉鲜肉 (水饺) x1 ¥16.90	纯肉鲜肉 (水饺) x1 ¥20.20
配送费 ¥3	配送费 ¥3
万家饺子 (软件园E18店) ▼ ¥19	万家饺子 (软件园E18店) ▼ ¥19
万家饺子 (软件园E18店) ▼ ¥19	万家饺子 (软件园E18店) ▼ ¥19

foodExplain	foodImg	foodPrice	businessId
猪肉馅饺子	data:image/png;base64,iVBORw0KGgoAAAANSU...EgAAAIIAAAC...	20.20	10001
玉米清香	data:image/png;base64,iVBORw0KGgoAAAANSU...EgAAAIIAAAC...	15.00	10001
三鲜馅饺子	data:image/png;base64,iVBORw0KGgoAAAANSU...EgAAAIIAAAC...	20.00	10001
三鲜馅饺子	data:image/png;base64,iVBORw0KGgoAAAANSU...EgAAAIIAAAC...	15.00	10001

原先后端获取历史订单信息中的商品的价格等信息直接从总食品表中获取而非每个订单下单时的数据。还有如配送费等商家可修改但用户订单需要展示曾经的值。修改数据库表结构，在订单表**orders**中添加订单配送费字段，在订单详细信息表**oderDetailed**中添加下单时的详细信息，如价格等。并使后端在创建订单时在新字段中存储相应值，在展示历史订单时从订单详细信息中获取价格等需要历史值的数据。

由于订单的时效性，后面的开发中在订单列表新增展示下单的时间：



四、项目测试

4.1 用户界面测试

在用户界面测试阶段，我们重点关注了页面的导航性、美观性和规范性，确保整体设计符合客户需求并具备良好的用户体验。测试范围涵盖了窗口运行状态

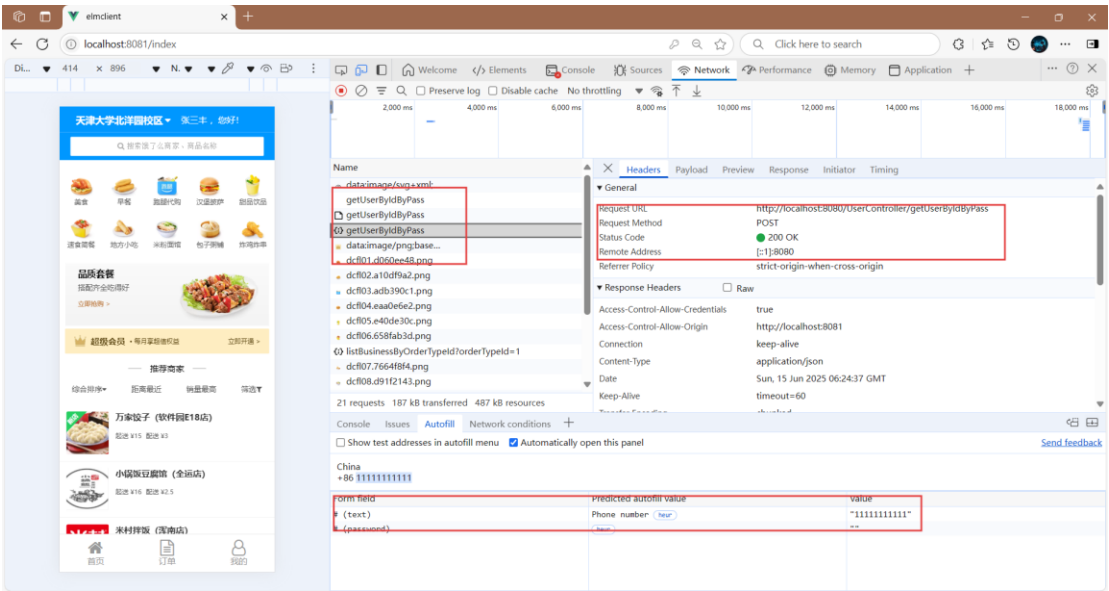
、提示信息的准确性、页面跳转的流畅性以及操作的便捷性等多个方面。我们的目标是验证界面风格的一致性，包括颜色搭配、字体选择、图标设计、标题展示等细节是否严格遵循了最初的设计规范。

测试过程中采用了通用的Web测试方法，主要通过手工测试和直观目测的方式进行。测试启动的前提条件是界面开发已经全部完成，具备完整的交互功能。最终的验收标准是界面必须达到可接受的质量水平，不仅要求视觉上的友好美观，更重要的是要确保操作的直观性和便利性，让用户能够自然而然地完成各项操作，完全符合日常使用习惯。整个测试过程注重细节把控，力求为用户提供一个既美观又实用的操作界面。

4.2 单元测试

单元测试主要针对程序中最小的功能单元——可能是结构化设计中的功能模块，也可能是面向对象设计中的类——进行验证，确保每个独立单元都能正确运行。

测试使用了浏览器的开发者工具，跟踪对每个功能单元前后端交互的情况，通过查看请求接口、传递参数、响应状态等信息测试单元是否达到预期效果。通过开发者工具可以更容易地通过响应码等信息判断错误类型，方便调试。



在本次实践中，我们在进行单元测试时首先检查每个组件模块的接口是否正确处理数据流，数据结构是否有变量未初始化、数据类型不匹配或者默认值设置不当等问题；然后重点测试程序的组件之间的跳转逻辑是否正确、逻辑判断是否合理、计算是否正确等。尤其要注意边界条件，比如刚好等于、大于或小于临界

值的情况，这些地方往往容易出问题。此外，还要看模块是否对常见异常情况做了合理处理，确保程序在遇到错误时能够正确应对，而不是直接崩溃或产生错误结果。

4.3 联合测试

联合测试是整个开发过程中非常关键的一环，像把一个个独立的零件组装成一台完整的机器。在单元测试确保每个模块都能正常工作之后，我们需要把这些模块按照预期设计拼装起来，看看它们能不能正常运作。这时候需要关注的是模块之间的配合问题——数据在接口处会不会莫名其妙丢失？某个模块的运行会不会给其他模块带来意想不到的麻烦？整个系统的数据结构能不能保持稳定？如果每个模块都带着一点点小误差，叠加起来会不会酿成大问题？

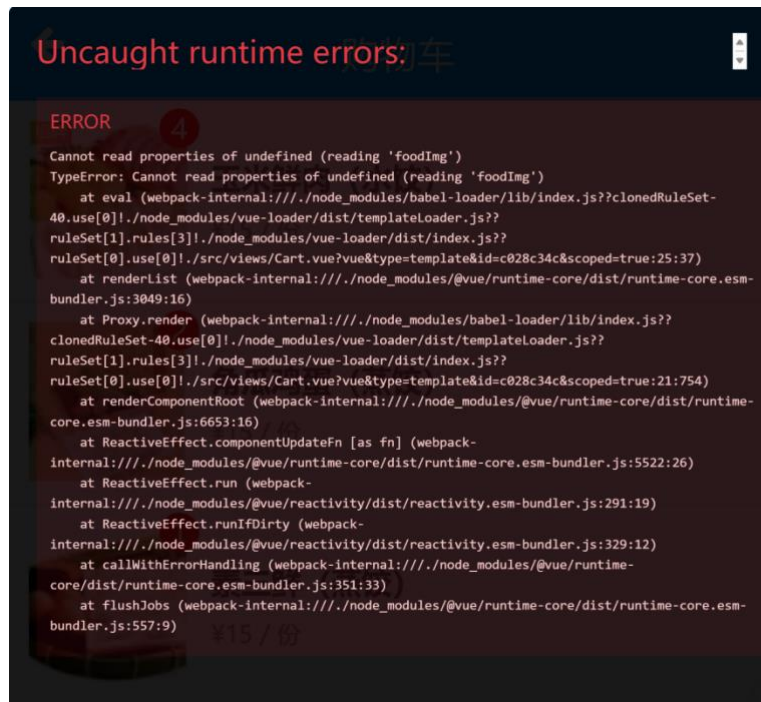
面对这些挑战，我们参考了两种组装策略：一口气把所有模块都组装起来来个"大联调"，或者像搭积木一样循序渐进，先组装核心模块，再逐步添加其他功能。这两种方式各有利弊，关键是要根据项目的具体情况来选择最合适的组装方式。说到底，联合测试就是要确保这些单独测试时表现良好的模块，组合在一起后依然能够稳定可靠地运转。

五、问题、解决对策与反思

问题1：点击购物车图标时报错



Cannot read properties of undefined (reading 'foodImg') 表示在尝试读取一个 undefined 对象的 foodImg 属性。



此时的代码：

```
<ul class="cart">
  <li v-for="item in cartItems" :key="item.foodId">
    <div class="cart-img">
      
      <div class="cart-img-quantity" v-show="item.quantity > 0">{{ item.quantity }}</div>
    </div>
    <div class="cart-info">
      <h3>{{ food[item.foodId].foodName }}</h3>
      <p>&#165;{{ food[item.foodId].foodPrice }} / 份</p>
    </div>
  </li>
</ul>
```

对策：使用 v-if 确保 food[item.foodId] 存在

```
<ul class="cart">
  <li v-for="item in cartItems" :key="item.foodId">
    <!-- 先检查 food[item.foodId] 是否存在 -->
    <template v-if="food[item.foodId]">
      <div class="cart-img">
        
        <div class="cart-img-quantity" v-show="item.quantity > 0">{{ item.quantity }}</div>
      </div>
      <div class="cart-info">
        <h3>{{ food[item.foodId].foodName }}</h3>
        <p>&#165;{{ food[item.foodId].foodPrice }} / 份</p>
      </div>
    </template>
    <!-- 可选：显示加载状态或默认内容 -->
    <template v-else>
      <div>加载中...</div>
    </template>
  </li>
</ul>
```

再次点击购物车图标，成功显示出购物车列表：



问题2：测试时发现推荐商家列表无法显示



对策：通过打开浏览器开发者工具发现前端期望 `{ list: [商家1, 商家2, ...] }`，但后端返回 `{ data: [...] }`，导致前端无法正确读取数据，修改前端的解析逻辑后解决了这个问题。

问题3：Vue 版本升级

项目文件的起始版本为Vue2，我们需要按照项目计划书要求将项目升级 为Vue3，以适应新的功能。升级过程终于到了很多的问题，诸如运行时报 错访问到未定义的变量，为此经过了非常久的调试找出了原因，即vue2和3对于v-if和v-for的渲染有先后顺序的区分；运行时一些script钩子函数的渲染时机与Vue2时不同导致页面无法正常显示。同时一些依赖包的安装和应用过成功出现了很多

的报错，诸如安装位置冲突造成新的包不能正常安装。同时兼容组件的使用报错也是层出不穷，在多次的调试和修改后升级为Vue3。

问题4：返回按钮功能优化

返回按钮我们采用的是小组件的方式，类似于源代码中的Footer.vue，但是随着开发的一步步发展，返回按钮的适用性得到了非常多的考验，为此我们对页面功能做了很多的调整，诸如设置跳转判断和禁用返回键。

反思：

在优化返回按钮功能的过程中，我们意识到简单的交互背后往往需要复杂的逻辑支撑。最初的小组件设计虽然便捷，但随着业务场景增多，暴露了跳转混乱等问题。通过调整跳转判断和禁用机制，我们提升了用户体验，也认识到基础功能需要持续迭代优化。

问题5：数据库sql脚本导入问题

在导入elm_admin.sql脚本时报错如下：

```
[3D000][1046] No database selected
摘要： 在 97毫秒中22/22 条语句已执行， 21条失败（文件中有 2,569 个符号）
```

但elm.sql导入没有问题，对比脚本内容，发现elm_admin.sql缺少create语句，修改如下：

```
CREATE DATABASE IF NOT EXISTS `elm_admin` /*!40100 DEFAULT CHARACTER SET utf8 */;
USE `elm_admin`;

SET FOREIGN_KEY_CHECKS=0;
-- -----
-- Table structure for admin
-- -----
DROP TABLE IF EXISTS `admin`;
CREATE TABLE `admin` (
```

成功导入后的数据库截屏：



	businessId	businessName	businessAddress	businessExplain	businessImg
1	10001	万家饺子（软件园E18店）	沈阳浑南区软件园E18楼	各种饺子	data:image/png;base64
2	10002	小锅饭豆腐馆（全运店）	沈阳市全运路126号	小锅套餐	data:image/png;base64
3	10003	麦当劳麦乐送（全运路店）	沈阳市全运路53号麦当劳	汉堡薯条	data:image/png;base64
4	10004	米村拌饭（浑南店）	沈阳市浑南区彩霞街26号	拌饭套餐	data:image/png;base64
5	10005	申记串道（中海康城店）	沈阳市中海康城社区48号	烤串炸串	data:image/png;base64
6	10006	半亩良田排骨米饭	沈阳市和平区文萃路58号	排骨米饭	data:image/png;base64
7	10007	茶兮鲜果饮品（国际软件园）	沈阳市国际软件园6号楼	甜品饮品	data:image/png;base64
8	10008	唯一水果捞（软件园E18店）	沈阳市软件园E18楼	新鲜水果	data:image/png;base64
9	10009	满园春饼（全运路店）	沈阳市全运路99号	春饼炒菜	data:image/png;base64
10	10010	海底	518hu	吃	data:image/png;base64

问题6：后端框架问题

项目课件提供的是基于Servlet的简易MVC架构，为简化配置和开发工作，并使用更多功能如自动依赖注入、简化的数据访问，小组成员在决定将其改为springbot架构，重写了所有后端接口。

修改步骤大致为：首先用@RestController和映射注解替换Servlet类，然后将Spring依赖注入管理服务层和DAO组件，再把JDBC操作改为MyBatis，最后用application.properties替代XML配置，并移除web.xml。

问题7：图片格式导致图片导入datagrip时失败

问题：在实现商家提交商户照片功能时，直接将图片以String格式导入数据库时失败。

对策：用户通过选择图片文件后，前端验证传入对象的类型和大小后，用FileReader对象来读取文件内容，并将其转换为Base64编码格式，最后提交到后端。

反思：前端必须用FileReader转换图片为Base64，避免直接存 File 对象；应校验图片格式和大小，防止非法数据提交。一个数据字段的导入需要前端正确转换格式、后端正确存储以及数据库字段匹配，缺一不可。

问题8：订单详情显示冲突

更改某食品的价格或名称时，会影响历史订单的相应信息，修改代码后，出现新问题：订单页面的历史数据会随新点开的订单信息更改。

例如：点击最近的订单，展示订单的信息为：三个纯肉鲜肉水饺，60元



再点击上一个43元的订单，结果这两个订单都显示了这一次点击的订单信息：

对策：

- 删除了全局的 `detaillet` 和 `index ref`
- 在 `detailletShow` 方法中，将获取的详情数据存储到当前订单对象 (`order.detaillet` 和 `order.index`)
- 在模板中，使用 `item.detaillet` 而不是全局的 `detaillet`
- 添加了缓存逻辑，如果已经加载过详情，就直接切换显示状态

这样修改后，每个订单的详情数据都是独立存储和显示的，点击不同订单时会显示各自对应的详情内容，不会再出现互相干扰的情况。

展示：

已支付订单信息:

万家饺子 (软件园E18店) ▼	¥23
纯肉鲜肉 (水饺) x2	¥20.00
配送费	¥3
小锅饭豆腐馆 (全运店) ▼	¥26.5
蛋黄焗豆花 x3	¥24.00
配送费	¥2.5
万家饺子 (软件园E18店) ▼	¥43
纯肉鲜肉 (水饺) x2	¥40.00
配送费	¥3
万家饺子 (软件园E18店) ▼	¥18.33
纯肉鲜肉 (水饺) x5	¥0.05
玉米鲜肉 (水饺) x1	¥15.28
配送费	¥3

反思: 在开发过程中,小组成员最初使用了全局变量来存储订单详情,导致不同订单的详情数据互相覆盖。这反映出我们对Vue响应式数据的理解还不够深入,没有充分考虑到组件状态的独立性。通过这次问题,我们认识到应该将数据与对应的UI元素紧密绑定,每个订单对象应当独立管理自己的状态,这样才能保证视图的正确性和数据的一致性。这种设计模式不仅解决了当前问题,也为后续维护和功能扩展打下了更好的基础。

问题9: 前后端请求方法不一致

前后端未约定好相应接口的请求method,前端使用POST调用接口,而后端接收query中的参数列表接收GET方法请求。导致获取数据失败,后端报错,浏览器出现405。

```
: Committing JDBC transaction on Connection [HikariProxyConnection@1675121340 wrapping com.mysql.cj.jdbc.ConnectionImpl@5d0e28b7]
: Releasing JDBC Connection [HikariProxyConnection@1675121340 wrapping com.mysql.cj.jdbc.ConnectionImpl@5d0e28b7] after transaction
: Resolved [org.springframework.web.HttpRequestMethodNotSupportedException: Request method 'POST' is not supported]
: Resolved [org.springframework.web.HttpRequestMethodNotSupportedException: Request method 'POST' is not supported]
```

通过统一调用方法,为了更强的安全性将不一致的接口统一使用参数包含在请求体里的POST方法,检查每个前后端接口,最终解决。